

Projekt: Krasnal

Eryk Daczyński
Małgorzata Maćkowiak
Jeremiasz Wacławski
Aleks Zieliński

May 18, 2026

Contents

1	Informacje wstępne	3
1.1	Treść projektu	3
1.2	temp	4
2	Funkcje	4
2.1	det()	4
2.2	EPS.h	5
2.2.1	cmpDouble()	5
2.2.2	sgn()	5
2.2.3	Kod źródłowy	5
2.3	is_point_on_segment()	5
2.4	do_segments_intersect()	6
2.5	distance()	6
2.6	is_point_in_polygon()	6
2.7	graham_scan()	7
3	Struktury	8
3.1	Point	8
3.2	Huffman.cpp	9
3.2.1	HuffmanNode	10
3.2.2	HuffmanCode	10
3.2.3	buildHuffmanTree()	10
3.2.4	generateHuffmanCodes()	11
3.2.5	Huffman()	11
3.2.6	HuffmanPrint()	11
3.2.7	HuffmanEquals()	11
3.2.8	HuffmanSort()	11
3.2.9	Kod źródłowy	11
4	Testy funkcji	13
4.1	test_det.cpp	13
4.2	test_is_point_on_segment.cpp	14
4.3	test_do_segments_intersect.cpp	15
4.4	test_is_point_in_polygon.cpp	16
4.5	test_graham_scan.cpp	17
5	Testy struktur	19
5.1	test_point.cpp	19
5.2	test_huffman.cpp	21
6	Prezentacja działania	23

1 Informacje wstępne

1.1 Treść projektu

Pracując nad poniższym projektem, należy: odfiltrować informacje istotne od nieistotnych sformalizować problem i podać jego specyfikację; do rozwiązania wykorzystać odpowiednie struktury danych oraz algorytmy; opisać teoretycznie rozwiązanie; przeanalizować jego optymalność oraz złożoność czasową/pamięciową; zaimplementować rozwiązanie w dowolnym języku programowania; przeprowadzić testy na różnych przykładach.

Problem

Odkąd królowa Śnieżka wraz z księciem z bajki rządzą krasnoludkami, ich królestwo nabiera siły i rośnie. Obecnie zamieszkuje je o wiele więcej niż tylko siedmiu krasnoludków. Każdy z nich jest specjalistą w swoim fachu i w kopalniach lub wyrobiskach potrafi wydobywać różne minerały. Królowa zna każdego ze swoich poddanych i wie jakie minerały każdy z nich potrafi wydobywać. To ona decyduje o miejscu pracy każdego z krasnoludków, jest to dla niej niełatwym zadaniem. Chcąc dbać o swoich obywateli, zapewniając im dostatek, królowa zleciła badania geologiczne w swoim kraju. Dzięki temu ma informacje o położeniu poszczególnych złóż w swoim kraju i wie, jaka jest ich wydajność (ilu krasnoludków może pracować w tym miejscu). Okazuje się, że najważniejsza jest miłość do pracy, którą się wykonuje. To znaczy, że jeśli krasnoludek pracuje przy materii zgodnej z jego preferencjami, wydobydzie kruszec o stałej wartości, niezależnie czy kopie złoto, węgiel, rudę miedzi, czy inny surowiec. Królowa Śnieżka poprosiła, abyś na podstawie posiadanych przez nią informacji pomógł jej przydzielić pracę każdemu z krasnoludków.

Po rozmowie z księciem okazało się, że jego ukochana bardzo dużo czasu spędza w kuchni, nadzorując gotowanie owsianki dla wygłodniałych brodaczy. Ilość potrzebnej owsianki zwiększa się, gdy krasnoludki muszą pokonać większą odległość z domu do pracy (można przyjąć, że jest to jedyny czynnik). Książę, chcąc ulżyć swojej wybrance, zmierzył i spisał odległości, pomiędzy każdym z wyrobisk i domkiem każdego z krasnoludków. Prosi, abyś uwzględnił te informacje. Przy podziale pracy chciałby, aby sumaryczna odległość, którą każdego dnia muszą pokonać krasnoludki była możliwie jak najmniejsza, przy czym nie możemy pozwolić, żeby spadła wartość produkowanych dóbr.

Niestety nie jest to jedyny problem. Królowa Śnieżka po incydencie ze złą królową jest nieco drażliwa, gdy pojawia się temat jabłek. Nakazała wyciąć wszystkie jabłonie w królestwie i zabroniła spożywania owoców. Krnąbrne krasnoludki starają się jednak wzbogacić swoją dietę o pyszne renety, antonówki, cortlandy, papierówki i ligole. Szmuglują przez granice królestwa pewne ilości tych owoców. W celu egzekucji prawa, zakazującego spożywania jabłek na terenie królestwa, książę codziennie rano musi okrążyć teren, na którym znajdują się wszystkie użytkowane kopalnie. Chciałby, żeby ta codziennie pokonywana odległość była możliwie mała. Gdy i to nie przynosiło skutku, książę wzdłuż trasy swego przejazdu rozstawił dekametrowców (krasnoludki uzbrojone w łuki ustawione co określoną liczbę metrów). Mają one zapobiec przerzucaniu jabłek przez granicę. Ataki takie wciąż zdarzają się często, atakowany jest dany wielometrowy odcinek granicy, przez który leci niezliczona liczba jabłek w jednym momencie. Dekametrowcy nie mogą opuścić swojego odcinka granicy, a najskuteczniejszą obroną przed lecącymi jabłkami okazuje się salwa.

Najgłośniejszy z krasnoludków atakowanej części granicy powinien wydać sekwencję poleceń:

„strzały na cięciwy
- naciągnąć cięciwy
- strzał!”.

Głos krasnoludka jest jego dumą i sprawą drażliwą, niełatwo więc przychodzi im ustalenie najgłośniejszego. Zaproponuj szybki sposób znajdowania dekametrowca z danego kawałka granicy, który winien wydać rozkazy.

Jak widać, królowa i książę muszą wykonać ogrom pracy, wykorzystując skomplikowane działania. Martwią się, że ta wiedza zaginie. Pomóż im zapisać cenną wiedzę w elektronicznych księgach, oszczędzając elektroniczny papier. Zatroszcz się, żeby potomni mogli szybko

wyszukiwać potrzebne informacje.

Co powinny przygotować zespoły studenckie

W ramach projektu należy przygotować rozwiązanie problemu przedstawionego powyżej.

Rozwiązanie powinno składać się z:

- dokumentacji
- plików programów
- opisu poprawności rozwiązania
- raportów przeprowadzonych testów

Wyniki pracy powinny być zapisywane z wykorzystaniem systemu Git (prowadzący powinien mieć dostęp do repozytorium). Projekt będzie oceniany zgodnie z informacjami przedstawionymi na USOS. Każda osoba będzie musiała wypełnić dwie ankiety: opisującą jej wkład w rozwiązanie problemu oraz oceniającą kompletność rozwiązania problemów przez zespół, do którego należy.

1.2 temp

```
1 int main(){
2     int i = 0;
3
4     return i;
5 }
```

Plik źródłowy 1: x example

2 Funkcje

2.1 det()

Funkcja pozwalająca ustalić położenie punktu p_3 względem wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) > 0$, to p_3 leży po lewej stronie wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) < 0$, to p_3 leży po prawej stronie wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) = 0$, to punkty p_1, p_2, p_3 są współliniowe

```
1 #include "det.h"
2
3 double det(Point p1, Point p2, Point p3) {
4     // 3 x 3
5     // D = (x2 - x1)(y3 - y1) - (y2 - y1)(x3 - x1)
6
7     /*
8         | 1 x1 y1 | = p1
9         | 1 x2 y2 | = p2
10        | 1 x3 y3 | = p3
11
12        D > 0 => p3 na lewo od p1p2
13        D < 0 => p3 na prawo od p1p2
14        D = 0 => p3 "na" p1p2 (wspolliniowy)
15
16     */
17
18     return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x);
19 }
```

Plik źródłowy 2: /Struktury/Funkcje/det.cpp

2.2 EPS.h

2.2.1 cmpDouble()

cmpDouble(a, b) sprawdza która liczba jest większa z uwzględnieniem błędu typu liczbowego double

2.2.2 sgn()

sgn(x) sprawdza znak liczby *x*, korzystając z *cmpDouble(x, 0.0)* [dokładniej, to sprawdza czy liczba jest większa od 0]

2.2.3 Kod źródłowy

```
1 #ifndef EPS_H
2 #define EPS_H
3
4 constexpr double EPS = 1e-9;
5
6 inline int cmpDouble(double a, double b) {
7     if (a < b - EPS) return -1;
8     if (a > b + EPS) return 1;
9     return 0;
10 }
11
12 inline int sgn(double x) {
13     return cmpDouble(x, 0.0);
14 }
15
16 #endif
```

Plik źródłowy 3: /Struktury/Funkcje/EPS.h

2.3 is_point_on_segment()

W tej funkcji sprawdzamy przynależność punktu *P* do odcinka p_1p_2

Pierw sprawdzamy czy punkt *P* jest współliniowy z $\overrightarrow{p_1p_2}$, jeżeli nie to nie należy do odcinka p_1p_2

Następnie jeżeli jest współliniowy, to sprawdzamy kolejno współrzędne *x* i *y* czy spełniają warunki odpowiednio:

$$\min(p_1.x, p_2.x) \leq P.x \leq \max(p_1.x, p_2.x)$$

$$\min(p_1.y, p_2.y) \leq P.y \leq \max(p_1.y, p_2.y)$$

Jeżeli spełnione są oba te warunki to punkt *P* należy do odcinka p_1p_2

```
1 #include "is_point_on_segment.h"
2
3 // Sprawdzenie czy punkt P należy do odcinka p1p2
4 bool is_point_on_segment(Point p1, Point p2, Point P)
5 {
6     // P należy gdy p1 p2 i P sa wspolliniowe -> D = 0
7     if(sgn(det(p1, p2, P)) != 0){
8         return false;
9     }
10    // P należy gdy min(x1,x2) <= x3 <= max(x1,x2)
11    // oraz max(y1, y2) <= y3 <= max(y1, y2)
12    bool x_ok = cmpDouble(std::min(p1.x, p2.x), P.x) <= 0 &&
13                cmpDouble(P.x, std::max(p1.x, p2.x)) <= 0;
14    bool y_ok = cmpDouble(std::min(p1.y, p2.y), P.y) <= 0 &&
15                cmpDouble(P.y, std::max(p1.y, p2.y)) <= 0;
16
17    return x_ok && y_ok;
18 }
```

Plik źródłowy 4: /Struktury/Funkcje/is_point_on_segment.cpp

2.4 do_segments_intersect()

Funkcja sprawdza czy odcinki p_1p_2 i p_3p_4 przecinają się

Obliczamy $D_1 = \det(p_1, p_2, p_3)$, $D_2 = \det(p_1, p_2, p_4)$, $D_3 = \det(p_3, p_4, p_1)$ i $D_4 = \det(p_3, p_4, p_2)$

Jeżeli $D_1 \cdot D_2 < 0$ i $D_3 \cdot D_4 < 0$ to p_1p_2 i p_3p_4 przecinają się

Jeżeli $D_1 = 0$ lub $D_2 = 0$ lub $D_3 = 0$ lub $D_4 = 0$ to sprawdzamy przynależności punktu do prostej (ponieważ $\overrightarrow{p_1p_2}$ i $\overrightarrow{p_3p_4}$ są współliniowe)

```
1 #include "do_segments_intersect.h"
2
3 // Sprawdzenie czy odcinki p1p2 p3p4 sie przecinaja
4 bool do_segments_intersect(Point p1, Point p2, Point p3, Point p4)
5 {
6     double d1 = det(p1,p2,p3);
7     double d2 = det(p1,p2,p4);
8     double d3 = det(p3,p4,p1);
9     double d4 = det(p3,p4,p2);
10
11     // Przecinaja sie gdy:
12     // WARUNEK 2: kazdy odcinek przecina prosta wyznaczona przez drugi odcinek
13     if((sgn(d1) * sgn(d2) < 0) && (sgn(d3) * sgn(d4) < 0)) return true;
14
15     // WARUNEK 1: ktorys z koncow odc. nalezy do drugiego odc.
16     // Czy P1 lezy na P3P4 || Czy P2 lezy na P3P4 || Czy P3 lezy na P1P2 || Czy P4
17     // lezy na P1P2
18     if(is_point_on_segment(p3, p4, p1) || is_point_on_segment(p3, p4, p2) ||
19        is_point_on_segment(p1, p2, p3) || is_point_on_segment(p1, p2, p4)) {
20         return true;
21     }
22     return false;
23 }
```

Plik źródłowy 5: /Struktury/Funkcje/do_segments_intersect.cpp

2.5 distance()

Funkcja oblicza kwadrat odległości punktu a od punktu b według metryki euklidesowej

```
1 #include "distance.h"
2
3 double distance(Point a, Point b) {
4     return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
5 }
```

Plik źródłowy 6: /Struktury/Funkcje/distance.cpp

2.6 is_point_in_polygon()

Funkcja implementuje algorytm z wykładu sprawdzający czy punkt p należy do listy wierzchołków (punktów *Point*) wielokąta *polygon*

Ustalmy $u = \min(x : x \in \text{polygon})$, $v = \max(x : x \in \text{polygon})$

Jeżeli $p.x < u$ lub $p.x > v$ to p nie leży w wielokącie *polygon*

Jeżeli p leży na dowolnej krawędzi wielokąta *polygon* to p leży w wielokącie *polygon*

Niech $q = (v + 1, p.y)$ i $l = 0$, i dla każdej krawędzi $p.y$ należy do domknięto-otwartego przedziału współrzędnych y tej krawędzi oraz pq ją przecina, to zwiększ l o

Jeżeli l jest nieparzyste, to punkt p leży w wielokącie *polygon*

```
1 #include "is_point_in_polygon.h"
2
3 bool is_point_in_polygon(Point p, const std::vector<Point>& polygon){
4     double u = polygon[0].x;
5     double v = polygon[0].x;
6     int n = polygon.size();
7     for(int i = 0; i < n; i++){
```

```

8      u = std::min(u, polygon[i].x);
9      v = std::max(v, polygon[i].x);
10  }
11
12  if((p.x < u) || (p.x > v)) return false;
13
14  int l = 0;
15
16  Point q = Point(v + 1, p.y);
17
18  for(int i = 0; i < n-1; i++){
19      if(is_point_on_segment(polygon[i], polygon[i+1], p)) return true;
20      if((p.y >= std::min(polygon[i].y, polygon[i+1].y) && (p.y < std::max(polygon
21 [i].y, polygon[i+1].y))){
22          if(do_segments_intersect(polygon[i], polygon[i+1], p, q)) l++;
23      }
24  }
25
26  if(l % 2 == 1) return true;
27  return false;
}

```

Plik źródłowy 7: /Struktury/Funkcje/is_point_in_polygon.cpp

2.7 graham_scan()

Funkcja implementuje algorytm Grahama do wyznaczenia otoczki wypukłej

Pierw sortujemy rosnąco punkty względem $p.y$, gdy $p.y$ taki sam to względem $p.x$

Następnie usuwamy duplikaty (te same punkty) ze zbioru

Jeżeli zbiór zawiera mniej niż 4 unikalne elementy to zwracamy ten zbiór

Następnie sortujemy zbiór rosnąco względem współrzędnych kątowych, kolejno usuwamy bliższe punkty z tą samą współrzędną kątową co dalszy punkt

Tworzymy stos i dodajemy do niego pierwsze trzy punkty oraz dla każdego kolejnego punktu p_i z wcześniej przygotowanej tablicy dopóki następnik i następnik następnika punktu p_i nie jest skretem w lewo to usuwamy element ze stosu, następnie dodajemy p_i

Ostatecznym wynikiem jest zbiór punktów otoczki wypukłej zbioru wejściowego

```

1  #include "graham_scan.h"
2
3  // https://en.wikipedia.org/wiki/Graham_scan
4  std::vector<Point> graham_scan(std::vector<Point> points){
5      /// KROK 1: USUNIECIE DUPLIKATOW I USTAWIENIE NAJBLIŻSZEGO
6      /// PUNKTU JAKO PIERWSZY (NAJPIERW NA Y, W RAZIE REMISU NA X)
7
8      std::sort(points.begin(), points.end(), [](const Point& a, const Point& b) {
9          int cmpY = cmpDouble(a.y, b.y);
10         if (cmpY != 0) return cmpY < 0;
11         return cmpDouble(a.x, b.x) < 0;
12     });
13
14     points.erase(std::unique(points.begin(), points.end(), [](const Point& a, const
15 Point& b) {
16         return cmpDouble(a.x, b.x) == 0 && cmpDouble(a.y, b.y) == 0;
17     }), points.end());
18
19     /// PRZYPADKI, GDY JEST PONIZEJ 3 PUNKTOW
20     if (points.size() < 3) {
21         if (points.size() == 0) return {};
22         if (points.size() == 1 || points.size() == 2) return points;
23     }
24
25     /// KROK 2: SORTUJEMY I DOSTOSOWUJEMY LISTE PUNKTOW ZE WZGLEDU NA WSPOLRZEDNE
26     KATOWE
27
28     /// NAJPIERW SORTUJEMY LISTE PUNKTOW

```

```

27  std::sort(points.begin() + 1, points.end(), [&](const Point& pi, const Point& pj)
28  {
29      double d = det(points[0], pi, pj);
30      if (sgn(d) == 0) {
31          int cmpDist = cmpDouble(distance(points[0], pi), distance(points[0], pj))
32          ;
33          if (cmpDist != 0) return cmpDist < 0;
34          int cmpX = cmpDouble(pi.x, pj.x);
35          if (cmpX != 0) return cmpX < 0;
36          return cmpDouble(pi.y, pj.y) < 0;
37      }
38      return sgn(d) > 0;
39  });
40
41  /// USUWAMY Z LISTY BLIZSZE PUNKTY Z TA SAMA WSPOLRZEDNA KATOWA CO DALSZY PUNKT
42  std::vector<Point> furtherPoints;
43  furtherPoints.push_back(points[0]);
44
45  for(unsigned int i = 1; i < points.size(); i++){
46      while(i + 1 < points.size() && sgn(det(points[0], points[i], points[i+1])) ==
47          0)
48          i++;
49      furtherPoints.push_back(points[i]);
50  }
51
52  points = furtherPoints;
53
54  /// KROK 3: Utworzenie stosu i dodanie 3 pierwszych punktów
55  if (points.size() < 3) return points;
56
57  std::vector<Point> stack;
58  stack.push_back(points[0]);
59  stack.push_back(points[1]);
60  stack.push_back(points[2]);
61
62  /// KROK 4: Ostateczne dobranie punktów otoczki
63  for (unsigned int i = 3; i < points.size(); i++) {
64      while (stack.size() >= 2 && sgn(det(stack[stack.size() - 2], stack.back(),
65          points[i])) <= 0) {
66          stack.pop_back();
67      }
68      stack.push_back(points[i]);
69  }
70  return stack;
71 }

```

Plik źródłowy 8: /Struktury/Funkcje/graham_scan.cpp

3 Struktury

3.1 Point

Struktura naszego punktu w układzie kartezjańskim, zawiera ona definicję oraz operatory do porównywania dwóch punktów oraz wykonywania działań na nich

Operator mniejszości: " $p_1 < p_2$ " sprawdza czy punkt p_1 na lewo od p_2 . Jeżeli mają tę samą współrzędną $p_1.x$, to sprawdza czy p_1 jest poniżej p_2

Operator większości: $p_1 > p_2$ sprawdza $p_2 < p_1$

Operator równości: $p_1 == p_2$ sprawdza czy współrzędne x i y są takie same

Operator nierówności: $p_1 \neq p_2$ [≠] zwraca przeciwieństwo $p_1 == p_2$

Operator mniejszy-równy: $p_1 \leq p_2$ [$\leq$] zwraca $p_1 < p_2 \vee p_1 == p_2$

Operator większy-równy: $p_1 \geq p_2$ [$\geq$] zwraca $p_1 > p_2 \vee p_1 == p_2$

Operator przypisania: $p_1 = p_2$ przypisuje punktowi p_1 odpowiednio $p_1.x = p_2.x$ oraz $p_1.y = p_2.y$
Wypisanie punktu: $<< p$ wypisuje punkt p w postaci $(p_1.x, p_1.y)$

```

1 #include "Point.h"
2
3 Point::Point(double x, double y) : x(x), y(y) {}
4
5 // Operator mniejszosci
6 bool Point::operator<(const Point& other) const {
7     int cmpX = cmpDouble(x, other.x);
8     if (cmpX != 0) {
9         return cmpX < 0;
10    }
11    return cmpDouble(y, other.y) < 0;
12 }
13
14 // Operator wiekszosci
15 bool Point::operator>(const Point& other) const {
16     return other < *this;
17 }
18
19 // Operator rownosci
20 bool Point::operator==(const Point& other) const {
21     return cmpDouble(x, other.x) == 0 && cmpDouble(y, other.y) == 0;
22 }
23
24 // Operator nierownosci
25 bool Point::operator!=(const Point& other) const {
26     return !(*this == other);
27 }
28
29 // Operator mniejszosci lub rownosci
30 bool Point::operator<=(const Point& other) const {
31     return (*this < other) || (*this == other);
32 }
33
34 // Operator wiekszosci lub rownosci
35 bool Point::operator>=(const Point& other) const {
36     return (*this > other) || (*this == other);
37 }
38
39 // Operator przypisania
40 Point& Point::operator=(const Point& other) {
41     if (this != &other){
42         x = other.x;
43         y = other.y;
44     }
45
46     return *this;
47 }
48
49 // Wypisanie punktu
50 std::ostream& operator<<(std::ostream& stream, const Point& p){
51     stream << "(" << p.x << ", " << p.y << ")";
52     return stream;
53 }

```

Plik źródłowy 9: /Struktury/Point.cpp

3.2 Huffman.cpp

Bardziej niżeli struktura to zbiór oddzielnych funkcji przy użyciu których korzystamy z algorytmu Huffmana do kodowania tekstu (Algorytm Huffmana)

Poniżej zawartość pliku nagłówkowego dla łatwiejszego zrozumienia kodu źródłowego

```

1 #ifndef HUFFMAN_H
2 #define HUFFMAN_H

```

```

3
4 #include <memory>
5 #include <queue>
6 #include <vector>
7 #include <string>
8 #include <iostream>
9 #include <algorithm>
10
11 struct HuffmanNode{
12     char c;
13     int freq;
14     bool isLeaf;
15
16     std::unique_ptr<HuffmanNode> left;
17     std::unique_ptr<HuffmanNode> right;
18
19     HuffmanNode() : c(0), freq(0), isLeaf(false), left(nullptr), right(nullptr) {}
20     HuffmanNode(char character, int frequency) : c(character), freq(frequency),
21         isLeaf(true), left(nullptr), right(nullptr) {}
22 };
23
24 struct HuffmanCode{
25     char c;
26     std::string code;
27
28     HuffmanCode(char character, const std::string& huffmanCode) : c(character), code(
29         huffmanCode) {}
30 };
31
32 std::unique_ptr<HuffmanNode> buildHuffmanTree(const std::vector<char>& C, const std::
33     vector<int>& F);
34 std::vector<HuffmanCode> generateHuffmanCodes(const std::unique_ptr<HuffmanNode>&
35     node, const std::string& prefix = "");
36 std::vector<HuffmanCode> Huffman(const std::vector<char>& C, const std::vector<int>&
37     F);
38 void HuffmanPrint(const std::vector<HuffmanCode>& codes);
39 bool HuffmanEquals(const std::vector<HuffmanCode>& a, const std::vector<HuffmanCode>&
40     b);
41 std::vector<HuffmanCode> HuffmanSort(const std::vector<HuffmanCode>& codes);
42
43 #endif

```

Plik źródłowy 10: /Struktury/Huffman.h

3.2.1 HuffmanNode

Struktura która przechowuje znak *c*, liczbę wystąpień *freq*, flagę czy jest liściem *isLeaf*, wskaźnik do lewego dziecka *left* oraz do prawego *right*
 Dodatkowo konstruktory do sprawnego tworzenia obiektów

3.2.2 HuffmanCode

Struktura która zawiera znak *c* oraz kod jego odpowiedni kod Huffmana *code*
 Dodatkowo konstruktory do sprawnego tworzenia obiektów

3.2.3 buildHuffmanTree()

Funkcja, która jak nazwa wskazuje, tworzy drzewo kodowań Huffmana
 Definiujemy funkcję porównywania struktur HuffmanNode która sprawdza która jest większa ”>” według pola *HuffmanNode.freq*
 Następnie sprawdzamy czy któraś z tablic wejściowych (tablica znaków lub tablica liczby wystąpień) jest pusta, jak tak to zwracamy pusty wskaźnik
 Tworzymy kolejkę priorytetową typu min z priorytetem *HuffmanNode.freq*

Tworzymy struktury *HuffmanNode* na podstawie danych wejściowych i dodajemy ich wskaźniki do kolejki priorytetowej

Dopóki rozmiar kolejki priorytetowej jest większy od 1 to wybieramy dwie struktury o najniższej wartości *freq* (najwyższy priorytet) jako odpowiednio *left* i *right* oraz tworzymy nowy węzeł z wskaźnikami do *left* i *right* i odkładamy go do kolejki

Po wyjściu z pętli jeżeli kolejka jest pusta to zwracamy pusty wskaźnik, a w przeciwnym wypadku zwracamy jedyny element kolejki który jest korzeniem drzewa Huffmana

3.2.4 generateHuffmanCodes()

Funkcja która rekurencyjnie na podstawie struktury *HuffmanNode* i podanego prefixu tworzy kodowanie Huffmana

Tworzymy tablicę *HuffmanCode* i następnie jeżeli dany element jest liściem to dodajemy znak *c* i jego kod *prefix* do tablicy kodów, jeżeli nie jest liściem, to dla lewego dziecka wykonujemy *generateHuffmanCodes* na tym dziecku z tym samym prefixem plus 0 i zwróconą wartość dodajemy do tablicy kodów, a dla prawego dziecka tak samo tylko wywołujemy funkcję *generateHuffmanCodes* od prawego dziecka i z prefixem plus 1 oraz dodajemy do tablicy kodów

Ostatecznie zwracamy tablicę kodów

3.2.5 Huffman()

Wywołuje funkcję *buildHuffmanTree* i na jej wyniku wywołuje funkcję *generateHuffmanCodes* i zwraca jej wynik

3.2.6 HuffmanPrint()

Wypisuje *HuffmanCode* w postaci *znak : kod*

3.2.7 HuffmanEquals()

Funkcja sprawdza czy dwa kody Huffmana są równe w sensie mają ten sam znak *c* i ten sam kod Huffmana

3.2.8 HuffmanSort()

Funkcja sortuje tablicę kodów Huffmana alfabetycznie według znaku *c*

3.2.9 Kod źródłowy

```

1 #include "Huffman.h"
2
3 std::unique_ptr<HuffmanNode> buildHuffmanTree(const std::vector<char>& C, const std::
  vector<int>& F){
4     auto cmp = [](const std::unique_ptr<HuffmanNode>& a, const std::unique_ptr<
  HuffmanNode>& b){
5         return a->freq > b->freq;
6     };
7
8     if((C.size() != F.size()) || C.empty() || F.empty()){
9         return nullptr;
10    }
11
12    std::priority_queue<std::unique_ptr<HuffmanNode>, std::vector<std::unique_ptr<
  HuffmanNode>>, decltype(cmp)> pq(cmp);
13
14    for(size_t i = 0; i < C.size(); i++){
15        pq.push(std::move(std::make_unique<HuffmanNode>(C[i], F[i])));
16    }
17
18    while(pq.size() > 1){

```

```

19     auto left = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));
20     pq.pop();
21     auto right = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));
22     pq.pop();
23
24     auto parent = std::make_unique<HuffmanNode>();
25     parent->freq = left->freq + right->freq;
26     parent->isLeaf = false;
27     parent->left = std::move(left);
28     parent->right = std::move(right);
29
30     pq.push(std::move(parent));
31 }
32
33 if(pq.empty()) return nullptr;
34 auto root = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));
35 pq.pop();
36
37 return root;
38 }
39
40 std::vector<HuffmanCode> generateHuffmanCodes(const std::unique_ptr<HuffmanNode>&
41 node, const std::string& prefix){
42     std::vector<HuffmanCode> codes;
43
44     if(node->isLeaf){
45         codes.emplace_back(node->c, prefix);
46     } else{
47         if(node->left) {
48             auto leftCodes = generateHuffmanCodes(node->left, prefix + "0");
49             codes.insert(codes.end(), leftCodes.begin(), leftCodes.end());
50         }
51         if(node->right) {
52             auto rightCodes = generateHuffmanCodes(node->right, prefix + "1");
53             codes.insert(codes.end(), rightCodes.begin(), rightCodes.end());
54         }
55     }
56
57     return codes;
58 }
59
60 std::vector<HuffmanCode> Huffman(const std::vector<char>& C, const std::vector<int>&
61 F){
62     auto root = buildHuffmanTree(C, F);
63
64     if(root == nullptr){
65         return {};
66     }
67
68     std::vector<HuffmanCode> huffmanCodes;
69     huffmanCodes = generateHuffmanCodes(root);
70
71     return huffmanCodes;
72 }
73
74 void HuffmanPrint(const std::vector<HuffmanCode>& codes){
75     for(const auto& code : codes){
76         std::cout<<code.c<<" : "<<code.code<<std::endl;
77     }
78 }
79
80 bool HuffmanEquals(const std::vector<HuffmanCode>& a, const std::vector<HuffmanCode>&
81 b){
82     if(a.size() != b.size()) return false;
83
84     for(size_t i = 0; i < a.size(); i++){
85         if((a[i].c != b[i].c) || (a[i].code != b[i].code)){
86             return false;
87         }
88     }
89     return true;
90 }

```

```

84     }
85 }
86
87     return true;
88 }
89
90 std::vector<HuffmanCode> HuffmanSort(const std::vector<HuffmanCode>& codes){
91     if(codes.empty()) return {};
92
93     std::vector<HuffmanCode> sortedCodes = codes;
94     std::sort(sortedCodes.begin(), sortedCodes.end(), [](const HuffmanCode& a, const
HuffmanCode& b){
95         return a.c < b.c;
96     });
97     return sortedCodes;
98 }

```

Plik źródłowy 11: /Struktury/Huffman.cpp

4 Testy funkcji

4.1 test_det.cpp

Sprawdzamy poprawność działania funkcji *det()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1 #include "../det.h"
2 #include<iostream>
3
4 bool test1(){
5     Point p1(0, 0);
6     Point p2(3, 1);
7     Point p3(1, 3);
8
9     const double expected = 8;
10
11     return (det(p1, p2, p3) == expected);
12 }
13
14 bool test2(){
15     Point p1(0, 0);
16     Point p2(3, 1);
17     Point p3(4, -1);
18
19     const double expected = -7;
20
21     return (det(p1, p2, p3) == expected);
22 }
23
24 bool test3(){
25     Point p1(0, 0);
26     Point p2(3, 1);
27     Point p3(6, 2);
28
29     const double expected = 0;
30
31     return (det(p1, p2, p3) == expected);
32 }
33
34 bool test4(){
35     Point p1(0, 0);
36     Point p2(4, 2);
37     Point p3(2, 1);
38
39     const double expected = 0;
40
41     return (det(p1, p2, p3) == expected);
42 }

```

```

43
44 bool test5(){
45     Point p1(0, 0);
46     Point p2(4, 2);
47     Point p3(6, 3);
48
49     const double expected = 0;
50
51     return (det(p1, p2, p3) == expected);
52 }
53
54 bool test6(){
55     Point p1(0, 0);
56     Point p2(4, 2);
57     Point p3(3, 3);
58
59     const double expected = 6;
60
61     return (det(p1, p2, p3) == expected);
62 }
63
64 int main(){
65     std::cout<<"Testy dla funkcji det:"<<std::endl;
66     std::cout<<"Test 1: "<<(test1())<<"OK":"ERROR"<<" (det = 8)"<<std::endl;
67     std::cout<<"Test 2: "<<(test2())<<"OK":"ERROR"<<" (det = -7)"<<std::endl;
68     std::cout<<"Test 3: "<<(test3())<<"OK":"ERROR"<<" (det = 0)"<<std::endl;
69     std::cout<<"Test 4: "<<(test4())<<"OK":"ERROR"<<" (det = 0)"<<std::endl;
70     std::cout<<"Test 5: "<<(test5())<<"OK":"ERROR"<<" (det = 0)"<<std::endl;
71     std::cout<<"Test 6: "<<(test6())<<"OK":"ERROR"<<" (det = 6)"<<std::endl;
72
73     return 0;
74 }

```

Plik źródłowy 12: /Struktury/Funkcje/Testy/test_det.cpp

```

1 #!/bin/bash
2
3 NAME="test_det.exe"
4
5 g++ -Wall -o $NAME test_det.cpp ../det.cpp ../../Point.cpp
6 ./ $NAME
7 rm $NAME

```

Plik źródłowy 13: /Struktury/Funkcje/Testy/run_test_det.sh

4.2 test_is_point_on_segment.cpp

Sprawdzamy poprawność działania funkcji *is_point_on_segment()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1 #include <iostream>
2 #include "../is_point_on_segment.cpp"
3
4 bool test1(){
5     Point p1(0,0), p2(4,2), p3(2,1);
6
7     const bool expected = true;
8
9     return (is_point_on_segment(p1,p2,p3) == expected);
10 }
11
12 bool test2(){
13     Point p1(0,0), p2(4,2), p4(6,3);
14
15     const bool expected = false;
16
17     return (is_point_on_segment(p1,p2,p4) == expected);

```

```

18 }
19
20 int main(){
21     std::cout<<"Testy dla funkcji is_point_on_segment:"<<std::endl;
22     std::cout<<"Test 1: " <<(test1()?"OK":"ERROR")<<" (on segment)"<<std::endl;
23     std::cout<<"Test 2: " <<(test2()?"OK":"ERROR")<<" (not on segment)"<<std::endl;
24
25     return 0;
26 }

```

Plik źródłowy 14: /Struktury/Funkcje/Testy/test_is_point_on_segment.cpp

```

1 #!/bin/bash
2
3 NAME="test_is_point_on_segment.exe"
4
5 g++ -Wall -o $NAME test_is_point_on_segment.cpp ../../Point.cpp ../det.cpp
6 ./$NAME
7 rm $NAME

```

Plik źródłowy 15: /Struktury/Funkcje/Testy/run_test_is_point_on_segment.sh

4.3 test_do_segments_intersect.cpp

Sprawdzamy poprawność działania funkcji *do_segments_intersect()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1 #include <iostream>
2 #include "../do_segments_intersect.cpp"
3
4 // Odcinki rozłącznie, nie przecinają się
5 bool test1(){
6     Point A(1, 1), B(4, 2), C(2, 4), D(5, 5);
7     const bool expected = false;
8
9     return (do_segments_intersect(A, B, C, D) == expected);
10 }
11
12 // Odcinki przecinają się klasycznie X
13 bool test2()
14 {
15     Point A(0,0), B(4,4), C(0,4), D(4,0);
16     return(do_segments_intersect(A,B,C,D) == true);
17 }
18
19
20 // Odcinki stykają się w jednym końcowym punkcie
21 bool test3()
22 {
23     Point A(0,0), B(2,2), C(2,2), D(4,0);
24     return(do_segments_intersect(A,B,C,D) == true);
25 }
26
27 // Odcinki współliniowe i nachodzące na siebie
28 bool test4()
29 {
30     Point A(0,0), B(4,0), C(2,0), D(6,0);
31     return(do_segments_intersect(A,B,C,D) == true);
32 }
33
34 // Odcinki współliniowe ale rozłączne
35 bool test5()
36 {
37     Point A(0,0), B(2,0), C(3,0), D(5,0);
38     return(do_segments_intersect(A,B,C,D) == false);
39 }
40

```

```

41 int main(){
42     std::cout<<"Testy dla funkcji do_segments_intersect:"<<std::endl;
43     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (do not intersect)"<<std::endl;
44     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (intersect)"<<std::endl;
45     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (touch at endpoint)"<<std::endl;
46     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (collinear overlapping)"<<std::
47     endl; // wspollionowe nachodzące
48     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (collinear disjoining)"<<std::endl
49     ; // wspolliniowe rozłączne
50     return 0;
51 }

```

Plik źródłowy 16: /Struktury/Funkcje/Testy/test_do_segments_intersect.cpp

```

1 #!/bin/bash
2
3 NAME="test_do_segments_intersect.exe"
4
5 g++ -Wall -o $NAME test_do_segments_intersect.cpp ../../Point.cpp ../det.cpp ../
6 is_point_on_segment.cpp
7 rm $NAME

```

Plik źródłowy 17: /Struktury/Funkcje/Testy/run_test_do_segments_intersect.sh

4.4 test_is_point_in_polygon.cpp

Sprawdzamy poprawność działania funkcji *is_point_in_polygon()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1 #include "../is_point_in_polygon.h"
2 #include<iostream>
3
4 bool test1(){
5     std::vector<Point> polygon = {Point(0, 0), Point(5, 1), Point(3, 3), Point(5, 5),
6     Point(2, 4), Point(0, 5)};
7     Point p = Point(3, 2);
8
9     const bool expected = true;
10
11     return (is_point_in_polygon(p, polygon) == expected);
12 }
13
14 bool test2(){
15     std::vector<Point> polygon = {Point(0, 0), Point(5, 1), Point(3, 3), Point(5, 5),
16     Point(2, 4), Point(0, 5)};
17     Point p = Point(4, 3);
18
19     const bool expected = false;
20
21     return (is_point_in_polygon(p, polygon) == expected);
22 }
23
24 int main(){
25     std::cout<<"Testy dla funkcji is_point_in_polygon:"<<std::endl;
26     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (in polygon)"<<std::endl;
27     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (not in polygon)"<<std::endl;
28
29     return 0;
30 }

```

Plik źródłowy 18: /Struktury/Funkcje/Testy/test_is_point_in_polygon.cpp

```

1 #!/bin/bash
2
3 NAME="test_is_point_in_polygon.exe"

```



```

4
5 g++ -Wall -o $NAME test_is_point_in_polygon.cpp ../is_point_in_polygon.cpp ../../
  Point.cpp ../is_point_on_segment.cpp ../det.cpp ../do_segments_intersect.cpp
6 ./ $NAME
7 rm $NAME

```

Plik źródłowy 19: /Struktury/Funkcje/Testy/run_test_is_point_in_polygon.sh

4.5 test_graham_scan.cpp

Sprawdzamy poprawność działania funkcji *graham_scan()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1 #include "../graham_scan.h"
2 #include "../distance.h"
3 #include <iostream>
4
5 // Test 1 - pusty zbior -> otoczka tez pusta
6 bool test1(){
7     std::vector<Point> points, expected;
8
9     points = {};
10
11     expected = {};
12     points = graham_scan(points);
13
14     if(points.size() != expected.size()) return false;
15
16     for(unsigned int i = 0; i < points.size(); i++){
17         if(points[i] != expected[i]) return false;
18     }
19
20     return true;
21 }
22
23 // Test 2 - jeden punkt
24 bool test2(){
25     std::vector<Point> points, expected;
26
27     points = {Point(0, 0)};
28
29     expected = points;
30     points = graham_scan(points);
31
32     if(points.size() != expected.size()) return false;
33
34     for(unsigned int i = 0; i < points.size(); i++){
35         if(points[i] != expected[i]) return false;
36     }
37
38     return true;
39 }
40
41 // Test 3 - dwa identyczne pkt -> jeden w otoczce
42 bool test3(){
43     std::vector<Point> points, expected;
44
45     points = {Point(0, 0), Point(0, 0)};
46
47     expected = {Point(0, 0)};
48     points = graham_scan(points);
49
50     if(points.size() != expected.size()) return false;
51
52     for(unsigned int i = 0; i < points.size(); i++){
53         if(points[i] != expected[i]) return false;
54     }

```

```

55     return true;
56 }
57
58 // Test 4 dwa rozne pkt + duplikat -> tylko unikalne pkt
59 bool test4(){
60     std::vector<Point> points, expected;
61
62     points = {Point(0, 0), Point(0, 0), Point(1, 2)};
63
64     expected = {Point(0, 0), Point(1, 2)};
65     points = graham_scan(points);
66
67     if(points.size() != expected.size()) return false;
68
69     for(unsigned int i = 0; i < points.size(); i++){
70         if(points[i] != expected[i]) return false;
71     }
72
73     return true;
74 }
75
76 // Test 5 - sprawdzenie poprawnego wyboru pkt
77 bool test5(){
78     std::vector<Point> points, expected;
79
80     points = {Point(0, 0), Point(1, 1), Point(2, 0), Point(3, 0)};
81
82     expected = {Point(0, 0), Point(3, 0), Point(1, 1)};
83     points = graham_scan(points);
84
85     if(points.size() != expected.size()) return false;
86
87     for(unsigned int i = 0; i < points.size(); i++){
88         if(points[i] != expected[i]) return false;
89     }
90
91     return true;
92 }
93
94 // Test 6 - poprawnosc pkt (wiekszy zbior)
95 bool test6(){
96     std::vector<Point> points, expected;
97
98     points = {
99         Point(0, 0), Point(1, 0), Point(2, 0), Point(2, 1), Point(4, 2),
100         Point(3, 4), Point(1, 3), Point(5, 1), Point(3, 0), Point(2, 2)
101     };
102
103     expected = {Point(0, 0), Point(3, 0), Point(5, 1), Point(3, 4), Point(1, 3)};
104     points = graham_scan(points);
105
106     if(points.size() != expected.size()) return false;
107
108     for(unsigned int i = 0; i < points.size(); i++){
109         if(points[i] != expected[i]) return false;
110     }
111
112     return true;
113 }
114
115 // Test 7 - punkty wspolliniowe -> tylko skrajne powinny zostac
116 bool test7()
117 {
118     std::vector<Point> points, expected;
119
120     points = {
121         Point(0,0), Point(1,1), Point(2,2),

```

```

123     Point(3,3), Point(4,4)
124 };
125
126     expected = {Point(0,0), Point(4,4)};
127     points = graham_scan(points);
128
129     if(points.size() != expected.size()) return false;
130
131     for(unsigned int i = 0; i < points.size(); i++){
132         if(points[i] != expected[i]) return false;
133     }
134
135     return true;
136 }
137
138 int main(){
139     std::cout<<"Testy dla funkcji graham_scan:"<<std::endl;
140     std::cout<<"Test 1: "<<(test1())?"OK":"ERROR"<<" (empty set)"<<std::endl;
141     std::cout<<"Test 2: "<<(test2())?"OK":"ERROR"<<" (1 element set)"<<std::endl;
142     std::cout<<"Test 3: "<<(test3())?"OK":"ERROR"<<" (2 element set)"<<std::endl;
143     std::cout<<"Test 4: "<<(test4())?"OK":"ERROR"<<" (3 element set)"<<std::endl;
144     std::cout<<"Test 5: "<<(test5())?"OK":"ERROR"<<" (4 element set)"<<std::endl;
145     std::cout<<"Test 6: "<<(test6())?"OK":"ERROR"<<" (10 element set)"<<std::endl;
146     std::cout<<"Test 7: "<<(test7())?"OK":"ERROR"<<" (collinear points)"<<std::endl;
147
148
149     return 0;
150 }

```

Plik źródłowy 20: /Struktury/Funkcje/Testy/test_graham_scan.cpp

```

1 #!/bin/bash
2
3 NAME="test_graham_scan.exe"
4
5 g++ -Wall -o $NAME test_graham_scan.cpp ../graham_scan.cpp ../../Point.cpp ../det.cpp
6     ../distance.cpp
7 ./$NAME
8 rm $NAME

```

Plik źródłowy 21: /Struktury/Funkcje/Testy/run_test_graham_scan.sh

5 Testy struktur

5.1 test_point.cpp

Sprawdzamy poprawność struktury *Point* poprzez sprawdzenie czy określone operatory poprawnie działają i wykonują to co chcemy

```

1 #include<iostream>
2 #include<sstream>
3 #include " ../../Point.h"
4
5 bool test1(){
6     Point A(1,2);
7     Point B(4,5);
8
9     const bool expected = false;
10
11     return ((A > B) == expected);
12 }
13
14 bool test2(){
15     Point A(1,2);
16     Point B(4,5);
17

```

```

18     const bool expected = true;
19
20     return ((A < B) == expected);
21 }
22
23 bool test3(){
24     Point A(1, 2);
25
26     try{
27         std::ostringstream str;
28         str<<A;
29         return true;
30     } catch(...){
31         return false;
32     }
33 }
34
35 bool test4(){
36     Point A(1, 2);
37     Point B(1, 2);
38
39     const bool expected = true;
40
41     return ((A == B) == expected);
42 }
43
44 bool test5(){
45     Point A(1, 2);
46     Point B(3, 4);
47
48     const bool expected = true;
49
50     return ((A != B) == expected);
51 }
52
53 bool test6(){
54     Point A(1, 2);
55     Point B(1, 2);
56
57     const bool expected = true;
58
59     return ((A <= B) == expected);
60 }
61
62 bool test7(){
63     Point A(1, 2);
64     Point B(0, 0);
65
66     const bool expected = true;
67
68     return ((A >= B) == expected);
69 }
70
71 bool test8(){
72     Point A(1, 2);
73     Point B(3, 4);
74     B = A;
75
76     const bool expected = true;
77
78     return ((A == B) == expected);
79 }
80
81 int main(){
82     std::cout<<"Testy dla klasy Point:"<<std::endl;
83     std::cout<<"Test 1: " <<(test1()? "OK": "ERROR")<<" (operator >)"<<std::endl;
84     std::cout<<"Test 2: " <<(test2()? "OK": "ERROR")<<" (operator <)"<<std::endl;
85     std::cout<<"Test 3: " <<(test3()? "OK": "ERROR")<<" (operator <<)"<<std::endl;

```

```

86     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (operator ==)"<<std::endl;
87     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (operator !=)"<<std::endl;
88     std::cout<<"Test 6: "<<(test6()?"OK":"ERROR")<<" (operator <=)"<<std::endl;
89     std::cout<<"Test 7: "<<(test7()?"OK":"ERROR")<<" (operator >=)"<<std::endl;
90     std::cout<<"Test 8: "<<(test8()?"OK":"ERROR")<<" (operator =)"<<std::endl;
91
92     return 0;
93 }

```

Plik źródłowy 22: /Struktury/Funkcje/Testy/test_point.cpp

```

1  #!/bin/bash
2
3  NAME="test_point.exe"
4
5  g++ -Wall -o $NAME test_point.cpp ../../Point.cpp
6  ./$NAME
7  rm $NAME

```

Plik źródłowy 23: /Struktury/Funkcje/Testy/run_test_point.sh

5.2 test_huffman.cpp

Sprawdzamy poprawność zbioru funkcji *Huffman* poprzez sprawdzenie czy funkcja *Huffman* zwraca poprawne wyniki (przy okazji sprawdzamy wszystkie zdefiniowane funkcje pomocnicze)

```

1  #include "../Huffman.h"
2  #include<iostream>
3  using namespace std;
4
5  bool test1(){
6      vector<char> characters = {'a', 'b', 'c', 'd', 'e', 'f'};
7      vector<int> frequencies = {45, 13, 12, 16, 9, 5};
8
9      vector<HuffmanCode> expectedCodes = {
10         {'a', "0"},
11         {'b', "101"},
12         {'c', "100"},
13         {'d', "111"},
14         {'e', "1101"},
15         {'f', "1100"}
16     };
17
18     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
19
20     return HuffmanEquals(huffmanCodes, expectedCodes);
21 }
22
23 bool test2(){
24     vector<char> characters = {'a', 'b', 'c', 'd', 'e', 'f'};
25     vector<int> frequencies = {1, 1, 1, 1, 1, 1};
26
27     vector<HuffmanCode> expectedCodes = {
28         {'a', "110"},
29         {'b', "00"},
30         {'c', "111"},
31         {'d', "01"},
32         {'e', "101"},
33         {'f', "100"}
34     };
35
36     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
37
38     return HuffmanEquals(huffmanCodes, expectedCodes);
39 }
40

```

```

41 bool test3(){
42     vector<char> characters = {'m', 'k', 'x', 'y', 'z'};
43     vector<int> frequencies = {18, 9, 2, 3, 7};
44
45     vector<HuffmanCode> expectedCodes = {
46         {'k', "10"},
47         {'m', "0"},
48         {'x', "1100"},
49         {'y', "1101"},
50         {'z', "111"}
51     };
52
53     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
54
55     return HuffmanEquals(huffmanCodes, expectedCodes);
56 }
57
58 bool test4(){
59     vector<char> characters = {};
60     vector<int> frequencies = {};
61
62     vector<HuffmanCode> expectedCodes = {};
63
64     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
65
66     return HuffmanEquals(huffmanCodes, expectedCodes);
67 }
68
69 bool test5(){
70     vector<char> characters = {'x', 'y', 'z'};
71     vector<int> frequencies = {};
72
73     vector<HuffmanCode> expectedCodes = {};
74
75     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
76
77     return HuffmanEquals(huffmanCodes, expectedCodes);
78 }
79
80 bool test6(){
81     vector<char> characters = {};
82     vector<int> frequencies = {2, 3, 5, 7};
83
84     vector<HuffmanCode> expectedCodes = {};
85
86     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
87
88     return HuffmanEquals(huffmanCodes, expectedCodes);
89 }
90
91 int main(){
92     std::cout<<"Testy dla funkcji huffman:"<<std::endl;
93     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (normal set)"<<std::endl;
94     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (same freq)"<<std::endl;
95     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (normal set)"<<std::endl;
96     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (both empty)"<<std::endl;
97     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (freq empty)"<<std::endl;
98     std::cout<<"Test 6: "<<(test6()?"OK":"ERROR")<<" (chars empty)"<<std::endl;
99
100     return 0;
101 }

```

Plik źródłowy 24: /Struktury/Funkcje/Testy/test_huffman.cpp

```

1 #!/bin/bash
2
3 NAME="test_huffman.exe"
4

```

```
5 g++ -Wall -o $NAME test_huffman.cpp ../../Huffman.cpp
6 ./ $NAME
7 rm $NAME
```

Plik źródłowy 25: /Struktury/Funkcje/Testy/run_test_huffman.sh

6 Prezentacja działania